Nombre del Alumno:

UNIVERSIDAD COMPLUTENSE MADRID

Examen de Sistemas Inteligentes

27 de junio de 2025

Instrucciones generales para realizar el examen:

- **Contesta a las preguntas digitalmente usando Word**. Puedes apoyarte en tantas hojas en sucio adicionales como quieras, pero las respuestas deben ir en el examen de forma digital.
- **El examen escrito se entrega en formato PDF para evitar modificaciones**. Para guardar el formato pdf ir a Archivo>exportar>crear pdf. **NO SE ACEPTAN ENTREGAS QUE NO ESTEN EN FORMATO PDF.**
- El tiempo para la realización del examen es de **3h**. Consta de dos partes, una teórica con problemas que se contestan en el Word de 5 puntos y una práctica que se contesta usando Júpiter Notebook. La entrega debe contener un zip con lo siguiente en la raíz del zip: El pdf con las respuestas de la parte teórica, el notebook con la implementación de la parte práctica y los datos que se necesiten (dataset). En Jupiter Notebook podéis escribir cualquier consideración de ejecución de la práctica, pero es requisito obligatorio que **la parte práctica sea ejecutable sin la intervención del profesor**.
- Haz la entrega en el campus con tu cuenta de la UCM. Por seguridad, **Pon tu nombre al zip para evitar problemas y pon también tu nombre en el documento pdf y en Júpiter notebook**.

Preguntas cortas de teoría. (2,5 p)

Contesta a las siguientes preguntas cortas de teoría. Cada pregunta vale 0.5 puntos.

1. Disponemos de un sistema de simulación del resultado de una guerra mundial. En este entorno se pueden conectar agentes que controlan las decisiones de los diferentes países. Pueden existir un número indefinido de bandos, aunque mínimo serán 2. Los países pueden unirse a un bando de los disponibles o crear uno nuevo y declararle la guerra a cualquier otro país. Si se declara la guerra a un país, y este pertenece a un bando, se declara la guerra a todo el bando. Los agentes disponen de diferente tipo de armamento para desplegar y necesitan

gestionar la adquisición de los recursos que necesitan para mantener el conflicto (petróleo, munición, tanques, aviones, etc) ¿Qué tipo de sistema se trata? Razona la respuesta.

Respuesta:

Es un sistema multiagente de conflicto colectivo ya que cualquiera puede atacar a cualquiera, por lo tanto, es un conflicto de todos contra todos, solo que algunos pueden agruparse en bandos debido a que puede que no tengan habilidad suficiente. Como los recursos que necesitan son finitos no solo pealan por ver quien gana si no por los propios recursos.

2. Queremos construir un agente que sea capaz de resolver el siguiente problema. Disponemos de N^2 primeros números naturales en una matriz cuadrada M(NN) de tal manera que las sumas tanto por las filas como por columnas coincidan. No se pueden repetir los números incluidos en la matriz ¿Cómo implementarías el agente? Modélalo brevemente.

Respuesta:

Es un algoritmo genético, donde tendremos un array de dimensión NxN. No tiene sentido en este caso la mutación como la conocemos ya que generaría multitud de individuos inválidos, en este caso tiene más sentido intercambiar el valor de dos elementos cualesquiera. En el cruce pasa lo mismo, existen diferentes algoritmos de cruce para permutaciones como Cruce parcialmente mapeado o Cruce de órdenes. Pero en este caso no se pide el tipo de cruce. Sólo se pide definir la representación del problema como un vector de NxN de longitud donde se colocan los números, la función de fitness sería el numero de filas que suman igual.

3. Disponemos de un procesador con dos núcleos de procesamiento, denominado núcleo A y núcleo B. Ambos son igual de rápidos. El procesador debe resolver un conjunto de tareas con un coste de: **T1=2,5, T2=3, T3=10.5, T4=8.25, T5=1,5 y T6=9** unidades de tiempo**. La unidad de tiempo viene definida por la velocidad del procesador.** El reloj del procesador determina cuando se ejecuta la instrucción por lo que una instrucción no puede ser ejecutada antes de tiempo y se mueve en stepts enteros.

Con las siguientes restricciones:

- T1 debe hacerse antes que T2

- T3 antes que T6

- T4 antes que T5 y T1

Se pretende resolver el problema usando A*. Dime una heurística que sea admisible.

Suma de los tiempos de cada tarea / 2. El tiempo total es desde que se ejecuta la primera o dos primeras tareas hasta que termina la última. Pero una tarea puede tardar en ejecutarse menos del tiempo de un ciclo, por lo que no podemos coger el valor de los tiempos si no redondeando hacia arriba.

4. Queremos crear un modelo de machine learning que prediga a partir de los datos extraídos de una analítica, si el paciente puede tener leucemia o no. El uso de este modelo es como apoyo a la toma de decisiones médicas en un hospital público. ¿Qué modelo de machine learning elegirías a priori? Razona la respuesta.

En este caso queremos algoritmos de caja blanca, por lo que decisión tree, KNN o random forest por este orden a priori son los indicados.

5. Disponemos de un dataset de imágenes de gatos de tamaño arbitrario y queremos construir un clasificador que sea capaz de identificar la raza del gato viendo una imagen. En el dataset simplemente tenemos imágenes, pero no sabemos qué tipo de raza son. Explica brevemente como afrontarías el problema planteado, explicando el tratamiento de datos, el modelo o modelos utilizado y cualquier otro procedimiento que consideres debes hacer para poder crear el clasificador

Como no tenemos la raza de los perros debemos utilizar aprendizaje no supervisado.
Por ejemplo: K-mean o clustering aglomerativo. Deberíamos estudiar cuantas clases de gatos hay, para determinar el número de clases si usamos k-Medias. Si no podemos hacerlo por fuerza bruta o con la técnica del codo.
Para tratar los datos habría que normalizar los datos escalando todas las imágenes a una resolución concreta (podríamos usar un Extractor de características) y también podríamos usar PCA para reducir el tamaño de la imagen o data aumentation en caso de tener pocas imágenes.

Problema a resolver (2,5 p)

Queremos desarrollar un agente que se encarga de resolver puzles del videojuego El Profesor Layton. Para el que no los conozca, es una serie de videojuegos originarios de la videoconsola Nintendo DS que han ido teniendo diferentes entregas en múltiples consolas (Nintendo 3Ds, Switch) y móviles. La idea es que los desarrolladores del juego quieren convertirlo en un juego multijugador, donde se compita por ver quien es el jugador más rápido en resolver un puzle. Como es posible que, al principio, no haya suficientes jugadores humanos, los desarrolladores han optado por crear un agente que sea capaz de resolver los puzles de forma automática. En concreto y como prueba de concepto para ver su viabilidad, quieren diseñar un agente que

resuelva el puzle de “El aparcamiento” (*Parking Lot Gridlock) que aparece en el videojuego: El profesor Layton y la Villa Misteriosa.*



El orgullo del Profesor Layton, el Laytonmóvil, se ha quedado atrapado en un aparcamiento lleno de coches y ya no puede salir. La cosa está tan mal que los coches solo pueden moverse hacia delante y hacia atrás. Se puede mover los coches tantas casillas como se desee, siempre que la situación del puzle lo permita.

En el juego multijugador, cada jugador resuelve el puzle de forma independiente y el que lo haga en menos pasos ganará. Si se hace en el mismo número de pasos, hay empate y ambos ganan. Así pues, los diseñadores han planteado que exista dos agentes, uno para el nivel fácil (Que llamaremos agente **Easy**), donde no es necesario obtener el mínimo número de pasos y otro de alto nivel (que llamaremos **Hard**) donde el agente debe resolver el problema en el menor número de pasos.

Se pide:

1. ¿Qué tipo de agente es Easy y que tipo de agente es Hard? Razona la respuesta. (0.5p)

Ambos son deliberativos basados en objetivos.

2. Modela uno de ellos usando REAS. En el entorno describe que tipo de entorno es y por qué (0.5p).

R: el numero de pasos que realiza el agente.

E: matriz, coches. Entorno es discreto, totalmente observable, determinista, secuencial, estático, individual.

S: Los coches y el tablero.

Actuadores: mover coches.

3. Modela el agente Hard con el formalismo que consideres más adecuado (1p)

A* una matriz donde se tachan con el mismo numero cada coche. Todos los coches ocupan dos unidades parecer ser, por lo que podríamos tener un vector de coches y cada coche que sea su posición X,Y.

Estado inicial el indicado en la figura. Estado final donde el coche de layton este fuera del parking.

Operadores: mover un coche tantas unidades como digamos. Podemos indicar Mover(Coche,casillas) donde casillas puede ser un valor desde -6 a 6 por cada coche.

Heurística: numero de coches que hay entre el de layton y la salida, + 1.

4. Si solo puedes mover los coches 1 casilla cada vez, ¿Qué modificaciones tendrías que hacer al apartado anterior? (0,5p)

En este caso modificaríamos los operadores sy habría poerador MoverPos(coche) y MoverNeg(Coche) y la heurística habría que sumar también las distancias entre el coche de layton y la salida así como el número mínimo de pasos que debo mover para quitar los coches de en medio.

Ejercicio práctico: (5 puntos)

Disponemos del dataset **cat_breeds_clean.csv** que se adjunta junto con el enunciado del examen.

Dicho dataset contiene información acerca de características de gatos y queremos predecir su raza **(Breed)**. Los campos del dataset son los siguientes:

- Breed: Raza del gato
- Age_in_years: años en años
- Age_in_months: años en meses
- Gender: Género
- Neutered_or_spayed: Castrado o esterilizado. True lo está, False no.
- Body_length: Longitud del cuerpo.
- Weight: peso.
- Fur_colour_dominant: color de pelo dominante
- Fur_pattern: Patrón del pelaje
- Eye_colour: color de ojos
- Allowed_outdoor: se le permite estar fuera de casa
- Preferred_food: comida preferida: seca o húmeda
- Owner_play_time_minutes: Tiempo que su dueño juega con ellos
- Sleep_time_hours: horas de sueño
- Country: Pais
- Latitude: Latitud
- Longitude: Longitud

Se desea construir varios modelos para estudiar el comportamiento de los mismos y elegir aquel que mejor clasifique.

Se pide:

1. Limpia el dataset y realiza las transformaciones necesarias para que los datos puedan ser aplicables a cualquier modelo de machine learning. Debes imputar los datos faltantes y no eliminar ninguna fila ni ningún atributo. Justifica en una celda de markdown las decisiones que has tomado. (1 punto)
2. Representa gráficamente los datos para mostrar la distribución de las clases usando un gráfico de dispersión, donde cada clase tenga un color diferente. Di si a priori son linealmente separables. (0.5 puntos)
3. Usa MLPClassifier de SKLearn para entrenar un modelo. Dicho modelo debe conseguir al menos un 96% de accuracy. (1 punto)
4. Usa el algoritmo KNN para conseguir una precisión del 95%. (1 punto)
5. Usa el algoritmo RandomForestClassifier para conseguir una precisión del 97%. (1 punto)
6. Dibuja las matrices de confusión y cualquier otra métrica que necesites para justificar con qué modelo te quedas. Justifica la respuesta. (0.5 puntos)